

Connected Services Platform Guide

1 Key concepts

Our platform is based on Apache Kafka, which is an open-source distributed event streaming platform.

1.1 Transport protocol

All the data is offered by the platform using the Kafka protocol.

More information can be found in the official Kafka Protocol Guide.

1.2 Events

An `event` is a single message in the Kafka stream.

In our case all the messages will be represented using a proprietary `Unified Data Format` created for the purpose in `Protobuf` for high efficiency, flexibility, and extendability.

See also Events and Kafka Partitions.

1.3 Topics

A `topic` in the Kafka language is a way to organize the `events`.

Following a `publish/subscribe` model, `events` can be written by publishing them in a certain `topic` and can be read by subscribing to the same `topic`.

See also Kafka Topics.

1.4 Producers

`Producers` are responsible for publishing the `events` and organize them in `topics`.

See also Kafka Producers.

1.5 Consumers

`Consumers` are clients subscribed to the `topics` and receiving the stream of `events`.

A `consumer` can be represented by multiple instances, therefore Kafka provides the concept of `consumer group`. All the instances sharing the same `consumer group` will have a shared access to the data, meaning that the `events` will be sent only once across the same group.

TomTom will provide a set of allowed group ids that can be used to consume the data.

See also Kafka Consumers and Consumer Groups.

2 On-boarding

2.1 Registration

To be able to access the service, you need an account on the TomTom Developer Portal and an API-Key for the "Connected Services API" product.

If you don't have an account yet, please register by visiting https://developer.tomtom.com/user/register. It is recommended to use a team-oriented email address rather than a personal email address.

Upon successful account creation, notify your designated Account Manager by sending them an email. This communication should include the email address associated with the newly created developer account.

TomTom will then proceed with the configuration of the requisite API-keys, which will be associated with the provided developer account. Once this configuration process is complete, you will receive a notification via email.

Once these steps are completed, you can access and manage your API-keys through the TomTom Developer Portal in the API & SDK Keys section.

2.2 Authentication

The platform supports OAuth2 authentication.

The API-Key created in the previous step will be used to connect to the OAuth2 token endpoint and fetch the OAuth2 token. With the token you can connect to the Kafka cluster.

Since the request is already authenticated using the API-Key, there's no need for `clientId` and `clientSecret`, although they are still mandatory in the configuration and can be set to any valid string.

In the next paragraph there's an example of client configuration that follows such authentication flow.

2.3 Test the connection

The quickest way to test the new topic and start reading content is to use `kafka-console-consumer`, a tool included in the Kafka package. See APACHE KAFKA QUICKSTART for instructions on how to use it.

Create a configuration file with:

```
echo "group.id=INSERT_CONSUMER_GROUP_HERE
security.protocol=SASL_SSL
sasl.mechanism=OAUTHBEARER
sasl.jaas.config=org.apache.kafka.common.security.oauthbearer.OAuthBearerLoginModule \
required clientId=\".\" clientSecret=\".\";
sasl.login.callback.handler.class=org.apache.kafka.common.security.oauthbearer.secured.\
OAuthBearerLoginCallbackHandler
sasl.oauthbearer.token.endpoint.url=https://api.tomtom.com/connected-services/oauth2/\
token?key=INSERT_APIKEY_HERE&apiVersion=1" > config-consumer.properties
```

Remember to replace the placeholders from the template above for the `group.id` and `key`. The latter is a secret and needs to be stored securely.

If you copy and paste the above script, pay attention to the line breaks. The final `config-consumer.properties` should have **6 lines in total**.

Finally test the connection with:

```
bin/kafka-console-consumer.sh --bootstrap-server kafka-bootstrap.prod.connected-
services.tomtom.com:9094 --topic INSERT_TOPIC_HERE --consumer.config config-consumer.properties
```

Again remember to replace the placeholder with the actual name of the `topic`.

The list of topics that are available can be retrieved with the following command:

```
bin/kafka-topics.sh --bootstrap-server kafka-bootstrap.prod.connected-services.tomtom.com:9094 --
list --command-config config-consumer.properties
```

3 Create a consumer application

Additional documentation on how to build a Kafka consumer application can be found here:

- How to build your first Apache KafkaConsumer application
- Optimizing Kafka consumers